

Trabalho 4 - Prova 1

Identificação do Aluno

- **Nome:** Rafael Lima Sant'Ana
- **Matrícula:** 251035659
- **Disciplina:** FGA0146 - ESTRUTURAS DE DADOS 1
- **Turma:** T04

Questão 1: Vetores, Matrizes e Ponteiros

Código-fonte

```
#include <stdio.h>

int main() {
    // a) Vetor de inteiros com todos os elementos em 0
    int vetor[10] = {0};
    for (int i = 0; i < 10; i++) {
        printf("vetor[%d] = %d\n", i, vetor[i]);
    }

    // b) Matriz 3x3 de inteiros com diagonal principal em 1
    int m[3][3] = {0};
    for(int i = 0; i < 3; i++) {
        m[i][i] = 1;
    }

    return 0;
}
```

Questão 2: Structs

Código-fonte

```
#include <stdio.h>
#include <string.h>
```

```

typedef struct
{
    char nome[50];
    float preco;
    int qtd;
} Produto;

int main() {
    Produto p1;

    strcpy(p1.nome, "Arroz");
    printf("%s\n", p1.nome);

    p1.preco = 29.99;
    printf("%f\n", p1.preco);

    p1.qtd = 10;
    printf("%d\n", p1.qtd);

    return 0;
}

```

Questão 3: Ponteiros e Structs

Código-fonte

```

#include <stdio.h>
#include <string.h>

typedef struct
{
    char nome[50];
    float preco;
    int qtd;
} Produto;

int main(){
    Produto p1;
    strcpy(p1.nome, "Arroz");

```

```

    p1.preco = 29.99;
    p1.qtd = 10;

    // a) Ponteiro apontando para a estrutura p1
    Produto *ptr = &p1;

    // b) Imprimindo usando o operador seta (->)
    printf("%s\n", ptr->nome);
    printf("%f\n", ptr->preco);
    printf("%d\n", ptr->qtd);
    return 0;
}

```

c) Explicação

A diferença é que o operador ‘->’ é utilizado para acessar um membro da struct via ponteiro e o operador ‘.’ também acessa um membro da struct, mas diretamente pela variável da struct.

Questão 4: Alocação Dinâmica de Memória

Código-fonte

```

#include <stdio.h>
#include <string.h>
#include <stdlib.h>

typedef struct
{
    char nome[50];
    float preco;
    int qtd;
} Produto;

int main(){
    int n = 0;
    printf("Qual o tamanho do seu array?\n");
    scanf("%d", &n);
    getchar();

    Produto *arr = (Produto *) malloc(n * sizeof(Produto));

```

```

    for (int i = 0; i < n; i++) {
        printf("Qual o nome do item (%d/%d)?\n", (i+1), n);
        fgets((arr+i)->nome, 50, stdin);
        (arr+i)->nome[strcspn((arr+i)->nome, "\n")] = '\0';

        printf("Qual o preco do item (%d/%d)?\n", (i+1), n);
        scanf("%f", &(arr+i)->preco);

        printf("Qual a quantidade do item (%d/%d)?\n", (i+1), n);
        scanf("%d", &(arr+i)->qtd);
        getchar();
    }

    printf("\n=== PRODUTOS CADASTRADOS ===\n");
    for (int i = 0; i < n; i++) {
        printf("Produto %d: %s | ", (i+1), (arr+i)->nome);
        printf("Preco: %.2f | Quantidade: %d\n", (arr+i)->preco, (arr+i)->qtd);
    }

    free(arr);
    return 0;
}

```

Justificativa de Liberação (Free)

Liberar a memória usando `free()` no final do uso do array dinâmico é essencial para limpar a memória das informações que não são mais úteis pro algoritmo, assim evitando problemas de vazamento de memória.

Questão 5: Recursão

Código-fonte

```

#include <stdio.h>
// a) Escreva uma função recursiva int fatorial(int n).
int fatorial(int n) {
    if (n <= 1) {
        return 1;
    }
}

```

```

    return n * fatorial(n - 1);
}
// b) Escreva uma função recursiva int soma_vetor(int v[ ], int tam).
int soma_vetor(int v[], int tam) {
    if (tam <= 0) {
        return 0;
    }
    return v[tam - 1] + soma_vetor(v, tam - 1);
}

int main() {
    return 0;
}

```

c) Complexidade de Tempo da Função soma_vetor

A complexidade de tempo é **$O(n)$** . A função chama a si mesma recursivamente n vezes (onde n é o parâmetro tam). Como a função chamada n vezes tem complexidade de tempo **$O(1)$** , então a complexidade de tempo da é **$O(n)$** .

Questão 6: Arquivos

Código-fonte

```

#include <stdio.h>
#include <stdlib.h>

typedef struct {
    char nome[50];
    float preco;
    int qtd;
} Produto;

int main() {
    FILE *write_file = fopen("produtos.txt", "w");

    if (write_file == NULL) {
        printf("Erro ao abrir o arquivo para escrita!\n");
        return 1;
    }
}

```

```

    Produto p1 = {"Teclado Mecanico", 149.90, 5};
    Produto p2 = {"Mouse Wireless", 129.50, 10};

    fprintf(write_file, "%s\n%.2f\n%d\n", p1.nome, p1.preco, p1.qtd);
    fprintf(write_file, "%s\n%.2f\n%d\n", p2.nome, p2.preco, p2.qtd);

    fclose(write_file);

    FILE * read_file = fopen("produtos.txt", "r");

    if (read_file == NULL) {
        printf("Erro ao abrir o arquivo para leitura!\n");
        return 1;
    }

    Produto produto;
    int contador = 1;

    printf("=== DADOS LIDOS DO ARQUIVO ===\n");
    while (fscanf(read_file, " %[^\\n] %f %d", produto.nome, &produto.preco, &produto.qtd) ==
3) {
        printf("Produto %d:\n", contador++);
        printf("Nome: %s\n", produto.nome);
        printf("Preco: R$ %.2f\n", produto.preco);
        printf("Quantidade: %d\n\n", produto.qtd);
    }

    fclose(read_file);

    return 0;
}

```

Questão 7: Notação Assintótica

a) Ordem de Crescimento

Ordenando a escala crescente de complexidade:

1. $O(1)$
2. $O(\log n)$

3. $O(n)$
4. $O(n \log n)$
5. $O(n^2)$

b) Análise de Complexidade da Busca Linear

- **Melhor Caso ($O(1)$):** Ocorre quando o alvo é o primeiro elemento do array. O laço for executa apenas uma iteração e a função é finalizada instantaneamente.
- **Pior Caso ($O(n)$):** Acontece quando o alvo é o último elemento ou não existe dentro do array. Dessa forma, a validação passará obrigatoriamente por todos os n itens (tamanho total do vetor) antes de acabar, resultando em um tempo de processamento linear.